```
Output:
int(4)
int(4)
```

…but not multi-byte safe:

```
$mbStr = "\u{1F920} kaboom!"; // a single emoji char, then a space,
                              then kaboom!
var_dump(bin2hex("\u{1F920}")); // bytes 0xf0 0x9f 0xa4 0xa0 in UTF-8
var_dump(strspn($mbStr, "\u{1F920}")); // returns 4 because of multi-
                                       byte char
var_dump(strcspn($mbStr, " ")); // also returns 4
var_dump(strspn($mbStr, "\x9f\xa0\xf0\xa4")); // also returns 4 despite
                                                scrambling the byte order
```

```
Output:
string(8) "f09fa4a0"
int(4)
int(4)
int(4)
```

These two functions simply analyze the input strings at the byte level; they'll handle a null byte, but there is no awareness of encoding. Although not shown here, the two functions also accept optional length and offset parameters, in case you need to restrict the search to a subset of the input string bytes.

Counting

Another common task is to find the length of a string or find the number of times a particular pattern occurs, for which PHP offers the following functions:

Table 8-11. *String counting functions*

`count_chars()`	count occurrences of byte values in a string
`strlen()`, `mb_strlen()`	return number of bytes in a string
`substr_count()`, `mb_substr_count()`	count occurrences of sub-string in a string

`count_chars()`